Faculty of Science, Engineering, and Technology

COS30017 Software Development for Mobile Devices

Individual Assignment: Assignment 2: Rent a Car with Intents

Student's Name: Nguyen Hoang Minh - 105710964

DriveEase: a car rental app built with Kotlin, Jetpack Compose and Android Intents

Student name	Nguyen Hoang Minh
Student ID	105710964
Tutor	Pham Dang Khoa
Submission date	November 2026
GitHub repository	https://github.com/nguyenhoangminh25251006/COS30017-Software-Development-for-Mobile-Devices (Assignment 2 code is in the ASM 2 folder)
Figma	https://www.figma.com/design/54FhaO2u7iPpxt5mYDEJw6/Untitled?node-id=1-2&p=f&t=0Td45SNyTUCpB5dD-0

Contents

1 Introduction
2 Planning and Development Process
3 User Stories and Use Cases
4 UI/UX Design
5 System Architecture and Implementation
6 Functionality and Technical Implementation
7 Testing and Debugging
8 Challenges, Explorations and Takeaways
9 Conclusion
10 References
Appendices A to D

List of figures

Figure 1 GitHub commit history for Assignment 2
Figure 2 CatalogueScreen in light mode, with annotated regions
Figure 3 CatalogueScreen in dark mode
Figure 4 Search and sort results
Figure 5 Favourites row, including a rented favourite
Figure 6 RentalScreen with slider and live total cost
Figure 7 Validation error banners (400 credit cap and insufficient balance)
Figure 8 Vietnamese localisation on both screens
Figure 9 CatalogueScreen in landscape: side-by-side header and dashboard
Figure 10 RentalScreen in landscape with a two-column layout
Figure 11 Layered architecture of DriveEase
Figure 12 Navigation flow between the two activities
Figure 13 Booking successful toast and "Active Rentals" section
Figure 14 Booking cancelled toast and car returned toast

Figure 15 Unit test results in the Gradle test report

1 Introduction

1.1 Project overview

DriveEase is an Android car rental app. A user browses five vehicles, compares their key details, saves favourites and rents a car using an in-app credit balance. The app is written in Kotlin with Jetpack Compose and Material Design 3, and it is split into two activities that talk to each other through explicit Intents: MainActivity (the catalogue) and RentalActivity (the booking form).
The selected Car is sent to RentalActivity as a Parcelable extra, and the booking result is returned with the Activity Result API. Vehicle, credit and favourite data live in memory in Kotlin StateFlow streams, so nothing is written to a database.
The app had four goals: a clean browsing experience on a phone, rentals that respect strict credit rules, quick ways to find a car (search, sort, favourites), and a design that works in light and dark mode. Business rules are kept out of the UI and covered by 27 unit tests.

1.2 Link to the unit learning outcomes

Outcome	How the project addresses it
ULO1 Mobile versus desktop/web development	Activity lifecycle and Intent-based navigation instead of page requests; Parcelable for passing objects between components; state held in a ViewModel so it survives configuration changes such as rotation; a runtime locale switch that does not restart the Activity.
ULO2 Designing within hardware limits	Horizontal swipe cards and large touch targets for one-handed use on a small screen; lazily composed lists (LazyRow) that only build visible items; true black dark theme that saves power on OLED screens; minimum SDK 24 to cover older devices.
ULO3 Build, test and debug with the standard toolkit	Android Studio, Gradle, Compose, Material 3 components, JUnit unit tests, Compose UI tests and a debugging log of five defects with root causes (Section 7.5).

1.3 Scope and limitations

DriveEase is a proof of concept, not a commercial platform. The limits below are deliberate and were chosen to keep the focus on Intents, Parcelable and state handling.

- Five predefined vehicles: Toyota Camry, Tesla Model 3, Hyundai Tucson, BMW 3 Series and Honda Civic.
- Every user starts with 500 credits, and a single rental may not cost more than 400 credits.
- Rental length is 1 to 7 days, and pickup is assumed to be immediate.
- All data is kept in memory and is lost when the app is closed. There is no backend and no real payment.

- The interface is available in English and Vietnamese.

2 Planning and Development Process

2.1 Development plan

I read the assignment requirements first and split them into ten phases. The order was chosen so that the logic with the most risk (validation and state) was built and tested before the screens that depend on it. Screens came second, and visual refinement came last.

Phase	Task	Expected outcome
1	Analyse requirements, check Gradle configuration	Feature list, constraints and SDK setup confirmed
2	Car data model with Parcelable	Vehicle attributes and Intent serialisation
3	Domain logic (CarRentalDomain)	Pure, testable validation, search and sort rules
4	In-memory repository (CarRentalRepository)	StateFlow-based state for credits, bookings and favourites
5	ViewModel (CarRentalViewModel)	UI state separated from repository logic
6	MainActivity and CatalogueScreen	Browsing, search, sort, favourites, launching the rental screen
7	RentalActivity and RentalScreen	Slider, credit validation, booking result
8	Dark mode and Vietnamese localisation	Two themes, two languages
9	Automated tests (27 cases)	Business rules and boundaries verified
10	UI refinement and final review	Swipeable cards, monochrome theme, final polish

2.2 Development environment

Component	Technology
IDE	Android Studio (Meerkat / Narwhal)
Language	Kotlin 2.2.10
SDK levels	Minimum API 24 (Android 7.0), target and compile API 37

Component	Technology
UI framework	Jetpack Compose with Material Design 3
Build system	Gradle with Kotlin DSL (build.gradle.kts)
Test device	Pixel 9 emulator (Android 15); API 24 emulator for backwards compatibility
Version control	Git and GitHub Classroom

2.3 Development timeline

Development began on 12 October 2026 and continued until the final commit on 7 November 2026. Git only records the five days on which work was pushed, so the timeline below reconstructs the actual build order and estimated hours. About 76 hours were spent in total, including design, implementation, testing, report preparation and the final rotation work.

Date	Work stage	Time	Outcome
12-13 Oct	Analysis and planning: requirements, feature list, Gradle setup and project scaffold	4 h	Scope fixed; project builds
14-16 Oct	Data model: Car (Parcelable), RentalBooking, SortOption and domain rules	8 h	Model and pure validation, search and sort functions complete
17-18 Oct	State layer: CarRentalRepository and CarRentalViewModel with StateFlow	6 h	Reactive in-memory state layer
19-20 Oct	Interface build: MainActivity, CatalogueScreen, RentalActivity and RentalScreen	10 h	Two-screen app with Intent navigation
20 Oct	Testing: 22 unit tests and Compose UI instrumentation tests	4 h	Core rules covered; UI tests in place
20 Oct	Documentation: report outline, test plan, implementation plan and GenAI disclosure	2 h	Report and test plan scaffolded

Date	Work stage	Time	Outcome
20 Oct	First visual pass: high-resolution images, layout overhaul and Next navigation fix	4 h	Improved catalogue; navigation bug fixed
21-24 Oct	Review and refactor: code clean-up and post-build fixes	3 h	Simpler code and cleaner repository
25 Oct-1 Nov	Design exploration: Figma screens for the monochrome direction	6 h	Figma reference ready to implement
2 Nov	Visual redesign: monochrome theme and true black dark mode	5 h	New look on both screens; OLED theme
2 Nov	Interaction: back navigation, specification panel, Popular Cars row and list reset	6 h	Swipeable cards with stable selection
3-5 Nov	Localisation and controls: Vietnamese strings, locale provider and ViewModel tests	6 h	Bilingual UI; four ViewModel tests
6 Nov	Final UI pass: buttons, controls and submission build	2 h	Stable build before rotation work
7 Nov	Landscape support: adaptive two-column layouts and state persistence	8 h	Side-by-side layouts; state survives rotation
7 Nov	Rotation control: full-sensor orientation, one-tap rotate button and final checks	2 h	Rotation works both ways; final build

2.4 Commit timeline and version control

The table below is the commit timeline: it records what was pushed to GitHub Classroom and the real commit dates. Commits were pushed in batches on 20 October, 22 October, 2 November, 6 November and 7 November, so a commit date is not the date the work started; the 20 October commits contain work done from 12 October onwards. The table lists the twelve main commits; the two housekeeping commits from 22 October and the merge commits are omitted. The test count in a commit message reflects the suite at that time (six ViewModel tests were added later).

Commit	Date	Description
2124384	20 Oct	Implement Assignment 2: core models, domain logic and UI
56b71bb	20 Oct	Add 22 unit tests and UI instrumentation tests
01afce3	20 Oct	Add assignment report support documentation
c29d2b2	20 Oct	Fix Next navigation reactivity bug, add high-res images, UI overhaul
b732dd4	2 Nov	Remove logo, fix reactive favouriteCars state
79436b7	2 Nov	Redesign with monochromatic theme and pitch-black dark mode
a7451fc	2 Nov	Add Back navigation, spec detail circles, description
e179a8e	2 Nov	Replace next/back buttons with horizontal swipeable Popular car cards
86ef3fd	2 Nov	Fix: reset car list on sort or search change
0d34599	6 Nov	Update buttons and UI controls
862e599	7 Nov	Add screen orientation support with adaptive landscape layout and state persistence
9f87f7b	7 Nov	Enable fullSensor orientation and add one-tap rotate button in top bar

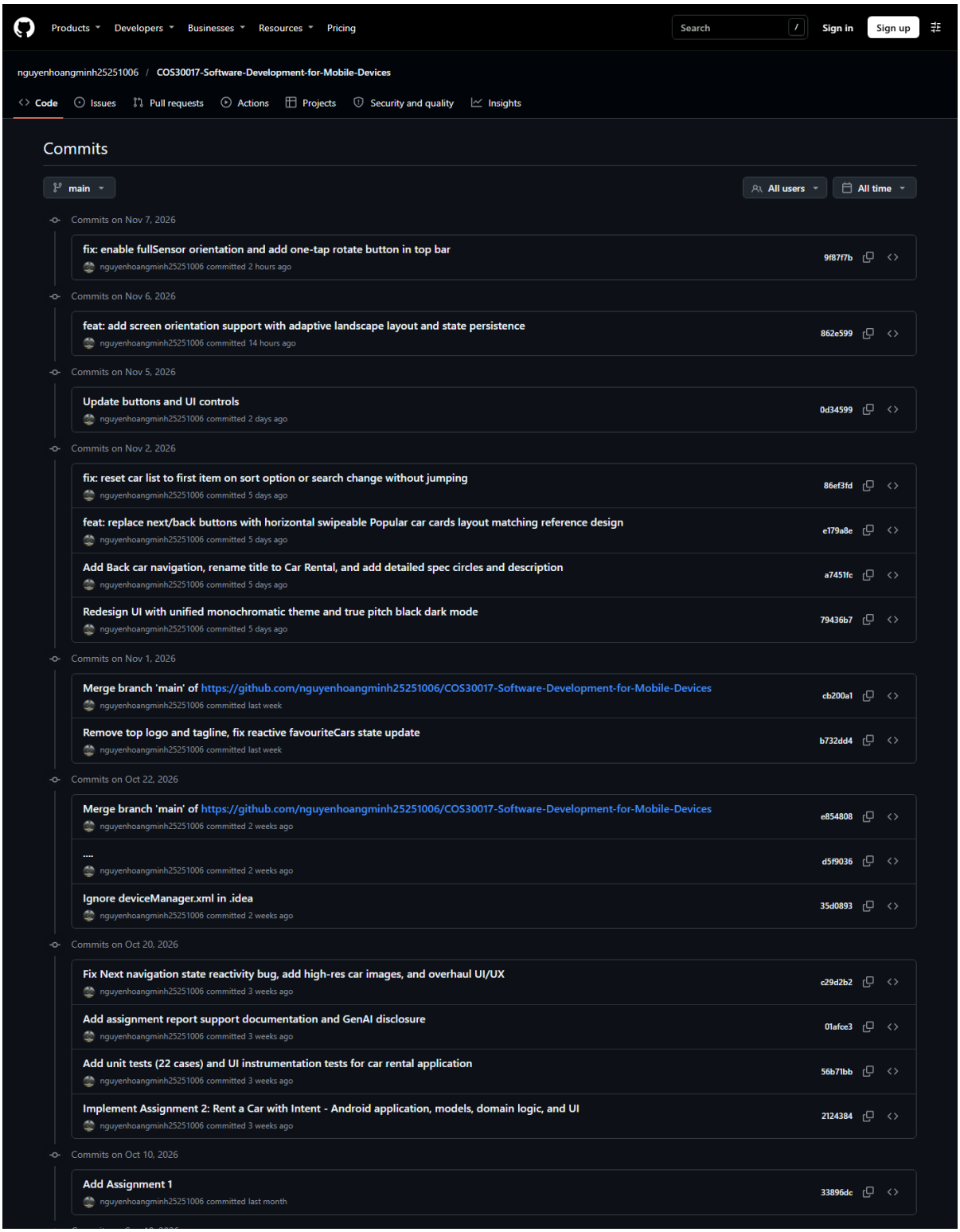


Figure 1. GitHub commit history for Assignment 2.

3 User Stories and Use Cases

3.1 User stories

User story 1: renting a vehicle. As a customer, I want to browse the available vehicles and rent one with my credit balance, so that I can pick a suitable car and complete a booking.
Acceptance criteria:

- The five vehicles can be browsed with the left and right arrows or by swiping the Popular Cars row.
- Each catalogue card shows the model, year, kilometres, star rating and daily cost.
- Rental length is chosen with a slider from 1 to 7 days, and the total cost updates as the slider moves.
- A booking is refused if the total exceeds 400 credits or the available balance.
- The user can confirm with "Confirm Booking", or cancel with "Cancel Booking" or the system Back button.
- After a successful booking the car leaves the catalogue, the credits are deducted and the car appears under "Active Rentals", where it can be returned for a full refund.

User story 2: finding and managing favourites. As a customer, I want to search, sort and save cars I like, so that I can quickly find vehicles that match my preferences.
Acceptance criteria:

- Search works on vehicle name and model through a text field.
- A heart icon on each card adds or removes a favourite, and favourites appear in their own Favourites row below the Popular Cars row.
- Sorting is available by rating (highest first), year (newest first) and daily cost (lowest first).
- A favourite stays visible after it has been rented.

3.2 Use case UC01: rent a vehicle

Actor	Customer
Precondition	The app is running and at least one vehicle is available.
Trigger	The customer taps "Rent" on a vehicle card.
Postcondition	The booking is recorded, credits are deducted and the vehicle is removed from the available list.

Main flow

1. The customer browses the catalogue and taps "Rent".
2. MainActivity starts RentalActivity with an explicit Intent carrying the Car (Parcelable), the credit balance, the dark mode state and the language.
3. RentalActivity shows the vehicle summary and the booking controls.
4. The customer moves the slider; the total cost updates live.
5. The customer taps "Confirm Booking".
6. CarRentalDomain.validateRental() checks that the car is free, the duration is 1 to 7 days, the total is at most 400 credits and the total does not exceed the balance.
7. On success RentalActivity returns RESULT_OK with the car ID and number of days.
8. MainActivity calls viewModel.processBookingResult(), which deducts credits and records the booking.
9. A localised toast confirms the booking and the amount deducted.

Alternative flows

- Cancel or Back: RentalActivity returns RESULT_CANCELED, nothing changes and a toast says no credits were deducted.

- Validation failure: an error banner appears inside RentalActivity and no booking is submitted until the customer shortens the rental or cancels.
- Car already rented: a validation error is returned immediately.

3.3 Use case UC02: search and manage favourites

Actor	Customer
Precondition	The catalogue contains at least one vehicle.
Trigger	The customer types in the search field, changes the sort option or taps a heart icon.
Postcondition	The list shows the filtered and sorted result and the favourites row is up to date.

Main flow

1. The customer types a keyword; CarRentalDomain.searchCars() filters name and model, ignoring case, on every keystroke.
2. The catalogue shows only matching vehicles and returns to the first car.
3. The customer taps the heart on a card; viewModel.toggleFavourite(carId) updates the favourites set.
4. The Favourites row updates at once.
5. The customer picks a sort option; CarRentalDomain.sortCars() returns a new sorted list and leaves the original data untouched.

4 UI/UX Design

4.1 Design principles

The interface follows an editorial, automotive style, similar to the restrained look of car brand apps. I used Nielsen's (1994) usability heuristics as a checklist while designing, and Material Design 3 components (Android Developers, n.d.-e) so that controls behave the way Android users expect. The high-fidelity design of both screens was prepared in Figma: https://www.figma.com/design/54FhaO2u7iPpxt5mYDEJw6/Untitled?node-id=1-2 (Nguyen, 2026).

Principle	How it is applied in DriveEase
Visibility of system status (Nielsen, 1994)	The credit balance sits in a card directly below the top bar on every catalogue view. On the rental screen the total cost changes as the slider moves, so the user sees the effect of every choice.
Error prevention	All rules are checked by validateRental() before any state changes. The slider is limited to 1 to 7 days, so an out-of-range duration cannot be typed.
Help users recognise and recover from errors	A banner with a warning icon states the exact problem, for example the cost and the 400 credit limit, and stays on screen while the user adjusts the slider.

Principle	How it is applied in DriveEase
Consistency and standards	The same monochrome palette and Material 3 components appear on both screens. Colour has fixed jobs: amber for ratings, red for favourites, charcoal or white for the main action.
Feedback	Toasts confirm bookings, cancellations, returns and favouriting in the current language.
Minimalist design	A card shows only name, model, year, kilometres, rating and daily cost, and the rental screen repeats this summary next to the booking controls instead of adding further detail.
Accessibility	Text uses sp units so it scales with system font size, icon buttons carry content descriptions, and Compose icon buttons keep a 48dp minimum touch target. Text colours on both themes are far above the 4.5:1 contrast ratio required by WCAG 2.1 level AA (W3C, 2018): white on pure black is 21:1.

4.2 Catalogue screen (MainActivity)

The screen is laid out from top to bottom so that the most frequent actions are within thumb reach and the credit balance never leaves the screen.

1. Top app bar: title, language menu (globe icon) and a light/dark Switch.
2. Credits card: the available credits used for validation and the number of cars ready to rent.
3. Search and sort: an OutlinedTextField for live search and three FilterChips (Rating, Newest, Lowest price) for one-tap sorting.
4. Popular Cars: a swipeable LazyRow of 280dp cards with previous/next arrows, an "n / total" counter and Back/Next Car buttons; each card shows the photo, name, daily cost, model, year, kilometres, star rating, a heart and a Rent button.
5. Favourites: a row of compact cards below Popular Cars, with a rented favourite marked "Currently Rented"; hidden when empty.
6. Active Rentals: one card per active booking showing the days, total cost and a Return button.

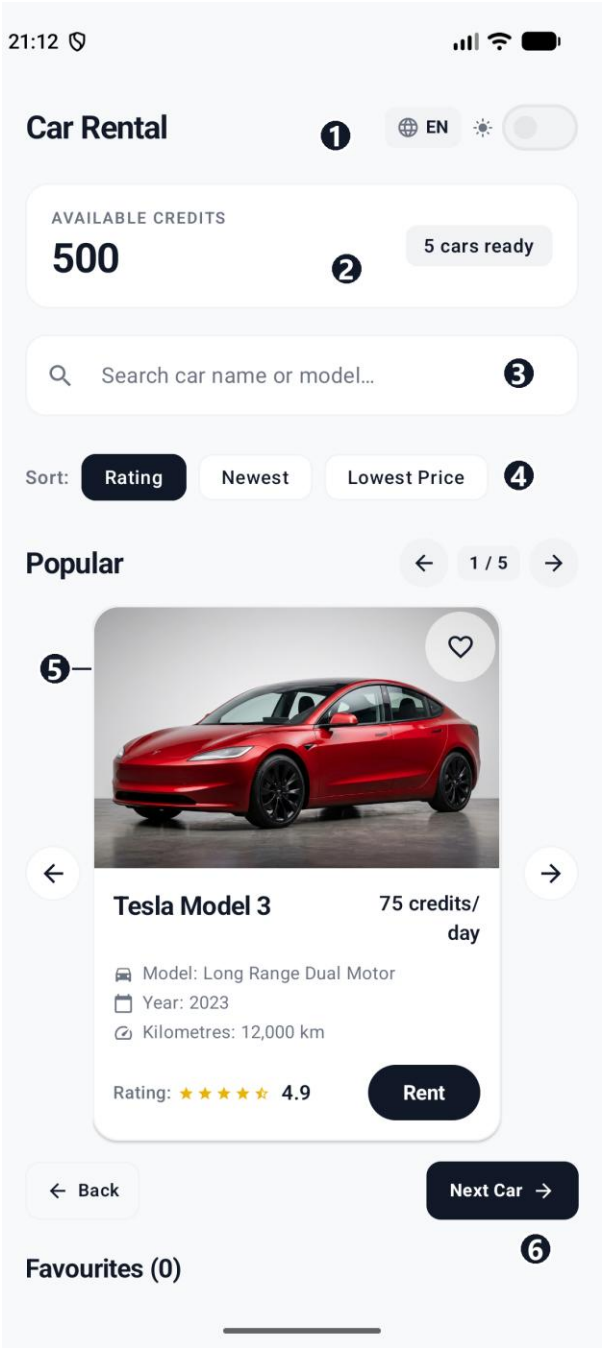


Figure 2. CatalogueScreen in light mode with annotated regions.

The widgets used on this screen are listed below. Two of them, the star rating bar and the switch, are not text widgets.

Widget	Purpose
TopAppBar	Title, language menu button and theme switch
Switch	Light and dark mode
IconButton (globe) and DropdownMenu	English and Vietnamese toggle
OutlinedTextField	Real-time search

Widget	Purpose
FilterChip (three chips)	One-tap sort selection
LazyRow with Card	Popular Cars and Favourites rows
Image	Vehicle photo
StarRatingBar (custom composable)	Rating with half-star precision
IconButton (heart, arrows)	Favourite toggle and previous/next navigation
Button	Rent, Back/Next Car and Return actions

4.3 Dark mode

The dark theme uses true black (#000000) for the background instead of the usual dark grey, because black pixels are switched off on OLED screens and save battery. Cards use #121214 so they still stand out from the background. The light theme uses an off-white background (#F8F9FA), white cards and deep charcoal (#111827) for primary actions. The selected mode is passed to RentalActivity so both screens always match.

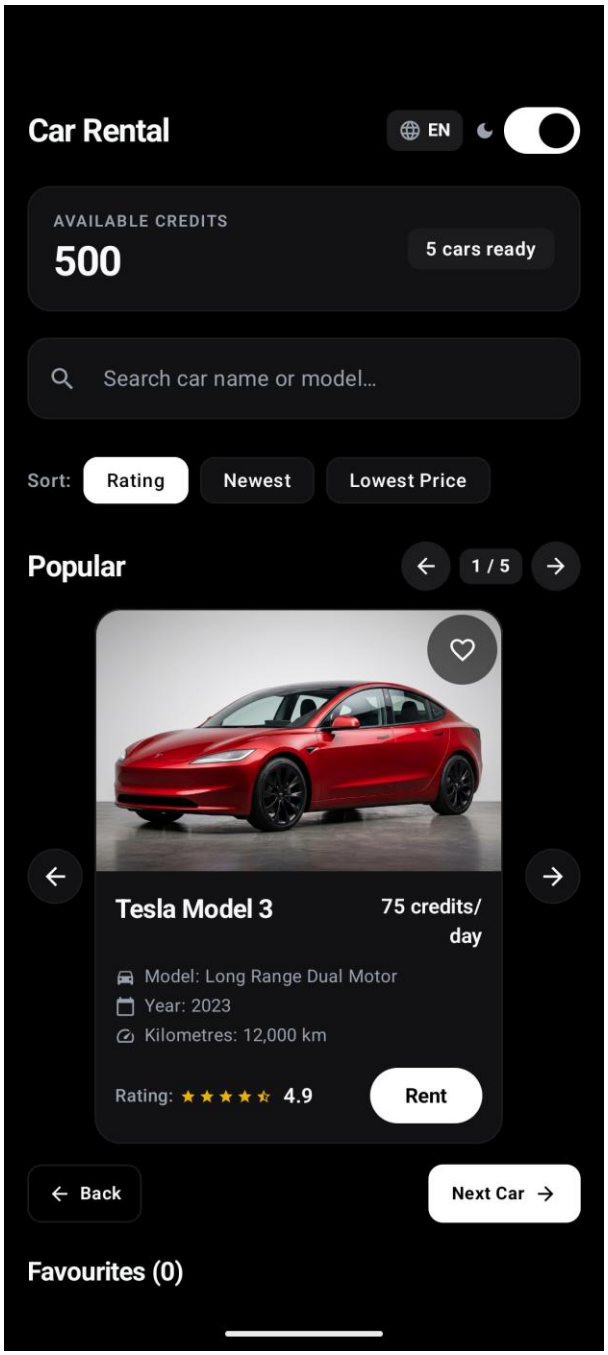


Figure 3. CatalogueScreen in dark mode.

4.4 Search, sort and favourites

Search results update on each keystroke, so the user gets feedback immediately and does not need a search button. Sorting never changes the underlying data, so clearing the search or changing the sort always returns the full list.

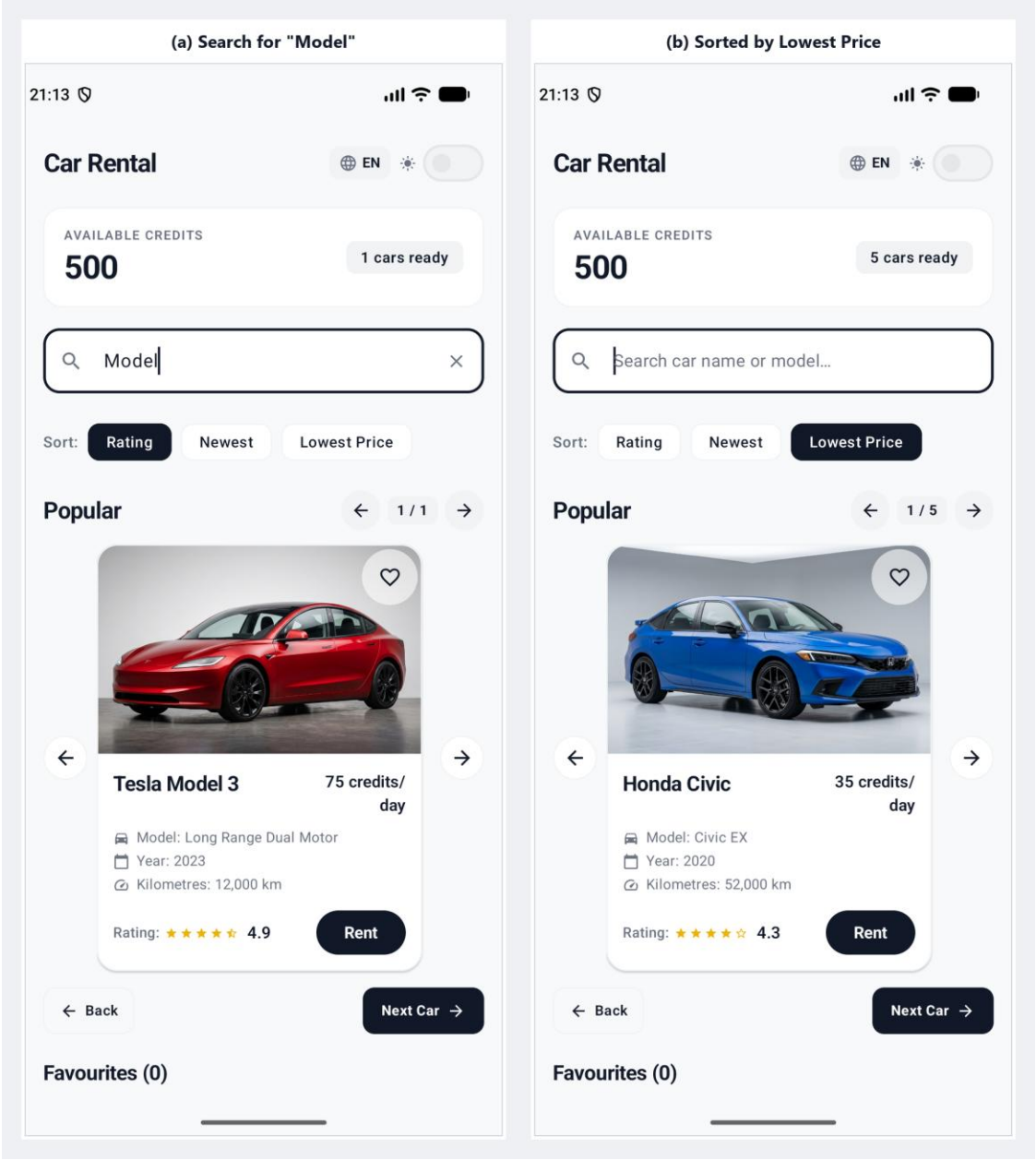


Figure 4. Search and sort results.

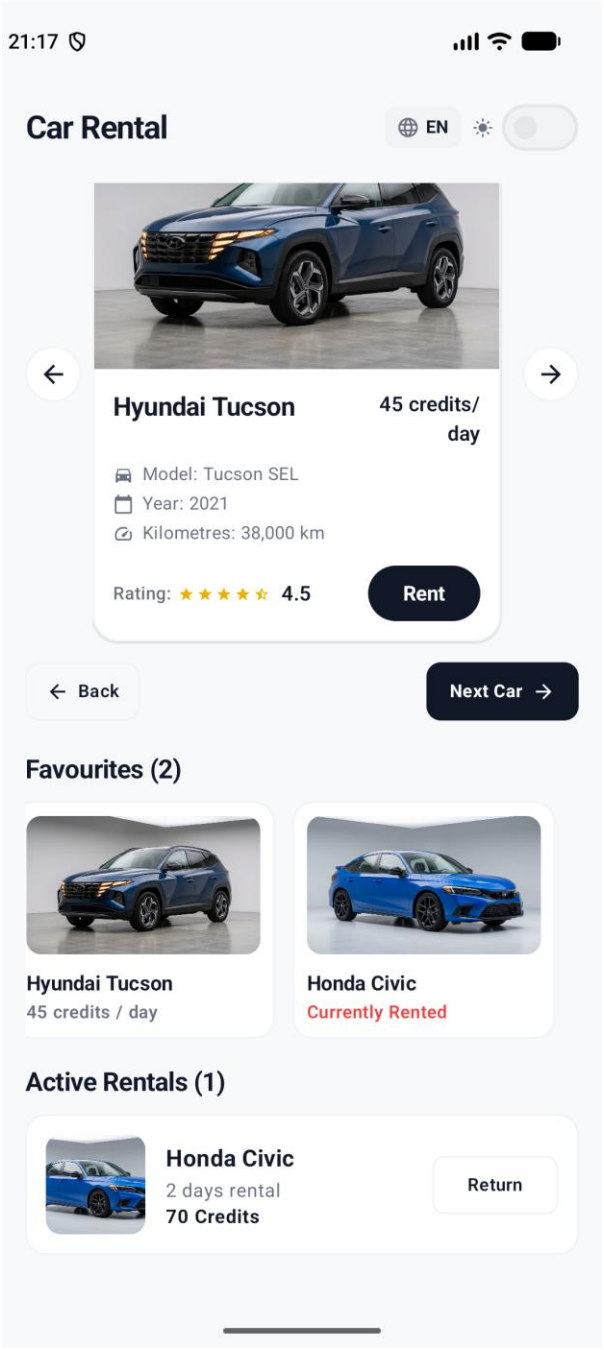


Figure 5. Favourites row, including a favourite that has been rented.

4.5 Rental screen (RentalActivity)

The rental form is a single scrolling column so that it fits any screen height: the vehicle image; name, model and year; a star rating; kilometres driven and daily cost; a duration card with the native Slider, a live day counter and the current balance and total cost; an error banner when validation fails; and two buttons side by side, "Cancel Booking" and "Confirm Booking".
The slider is a better fit than a number field on a phone: it needs no keyboard, it cannot produce an invalid number, and the cost updates while the finger is still moving (Android Developers, n.d.-g). The error

banner is placed under the slider so the user can fix the problem without leaving the form.

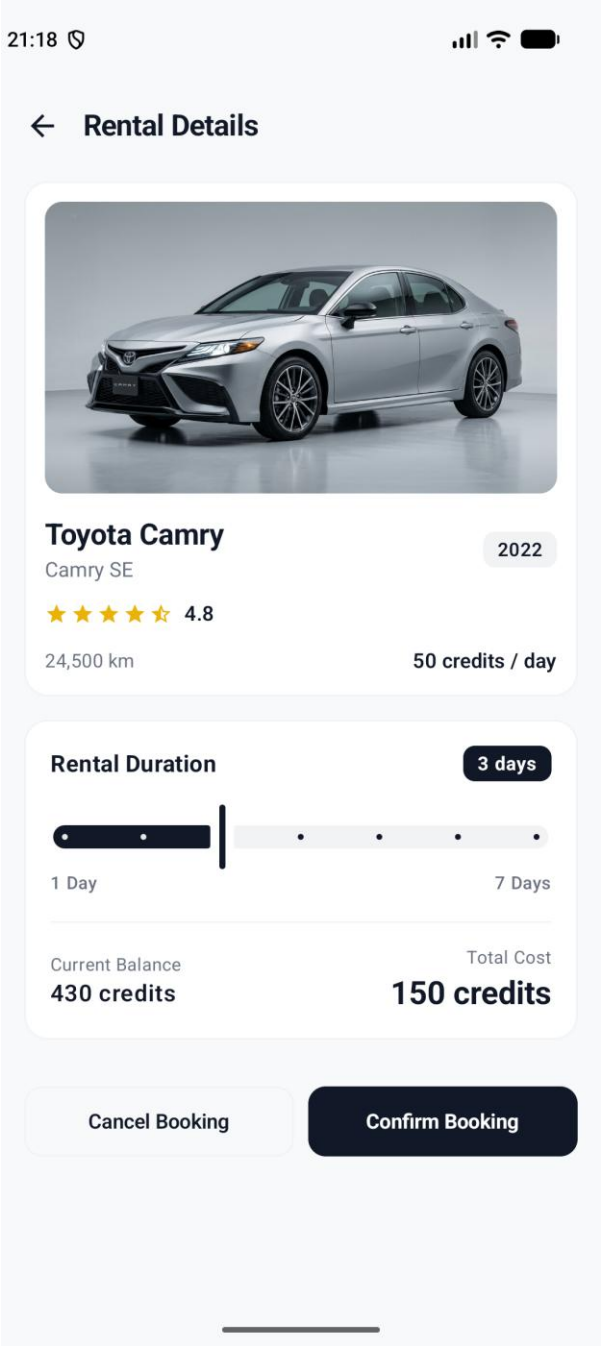


Figure 6. RentalScreen with slider and live total cost.

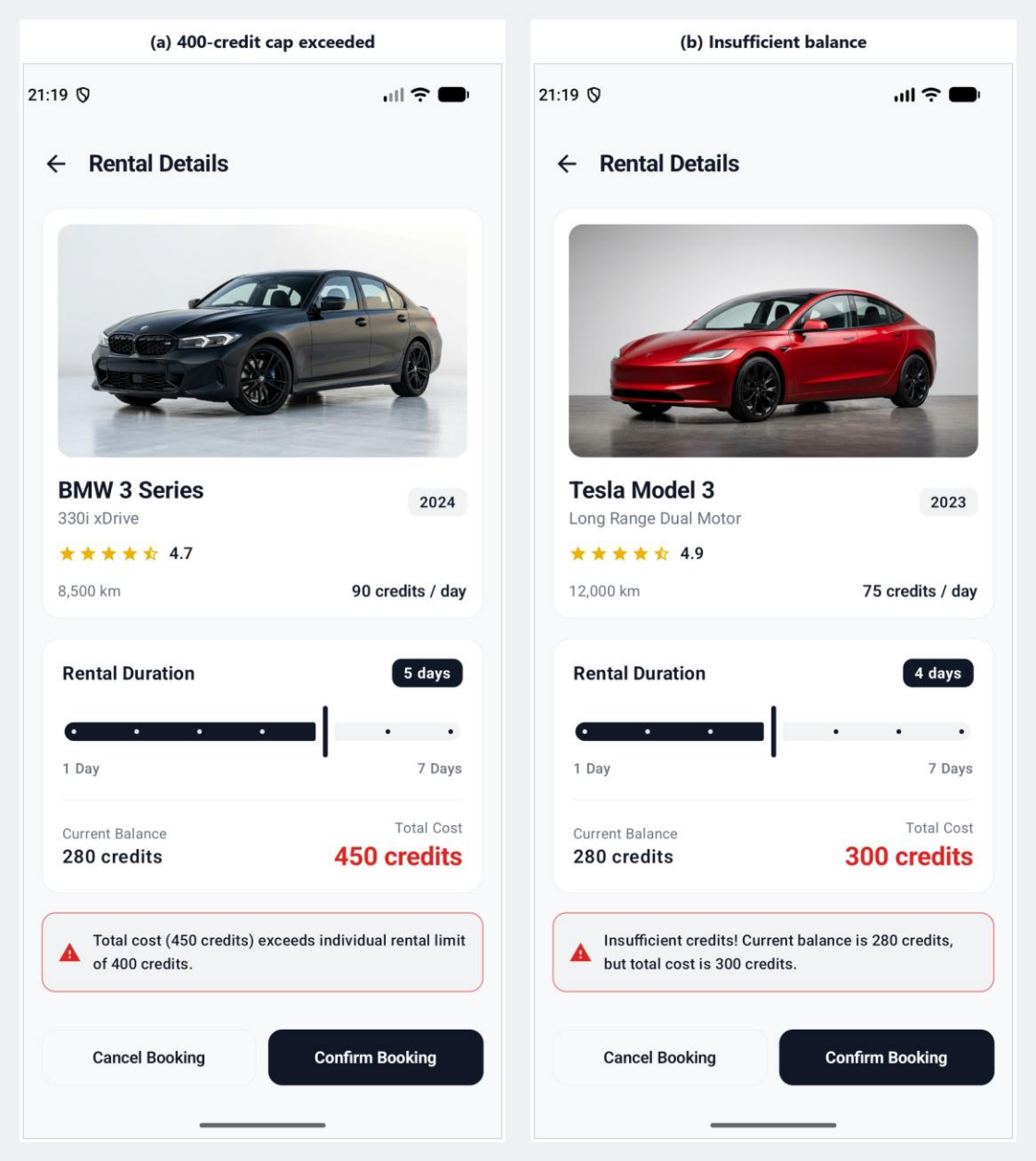


Figure 7. Validation error banners.

4.6 Designing for mobile constraints

- **Screen size:** one vehicle is the focus at a time; the rest are reached by swiping a horizontal row, which suits one-handed use. Secondary detail is moved to the rental screen instead of being squeezed onto the catalogue.
- **Memory:** LazyRow composes only the cards that are visible, and the five vehicles are held once in a canonical list. Search and sort create derived lists and do not copy images.
- **Battery:** true black dark mode, as described above.

Lifecycle: the ViewModel and rememberSaveable keep the selected car, slider position, balance, favourites and bookings when the device rotates; Section 4.8 describes the adaptive landscape layout.

- **Device range:** minimum SDK 24 with a version check for the newer getParcelableExtra overload.

4.7 Bilingual interface

The globe button switches the whole interface, including toasts, between English and Vietnamese without restarting the Activity. The choice is carried to RentalActivity in the Intent so both screens use the same language.

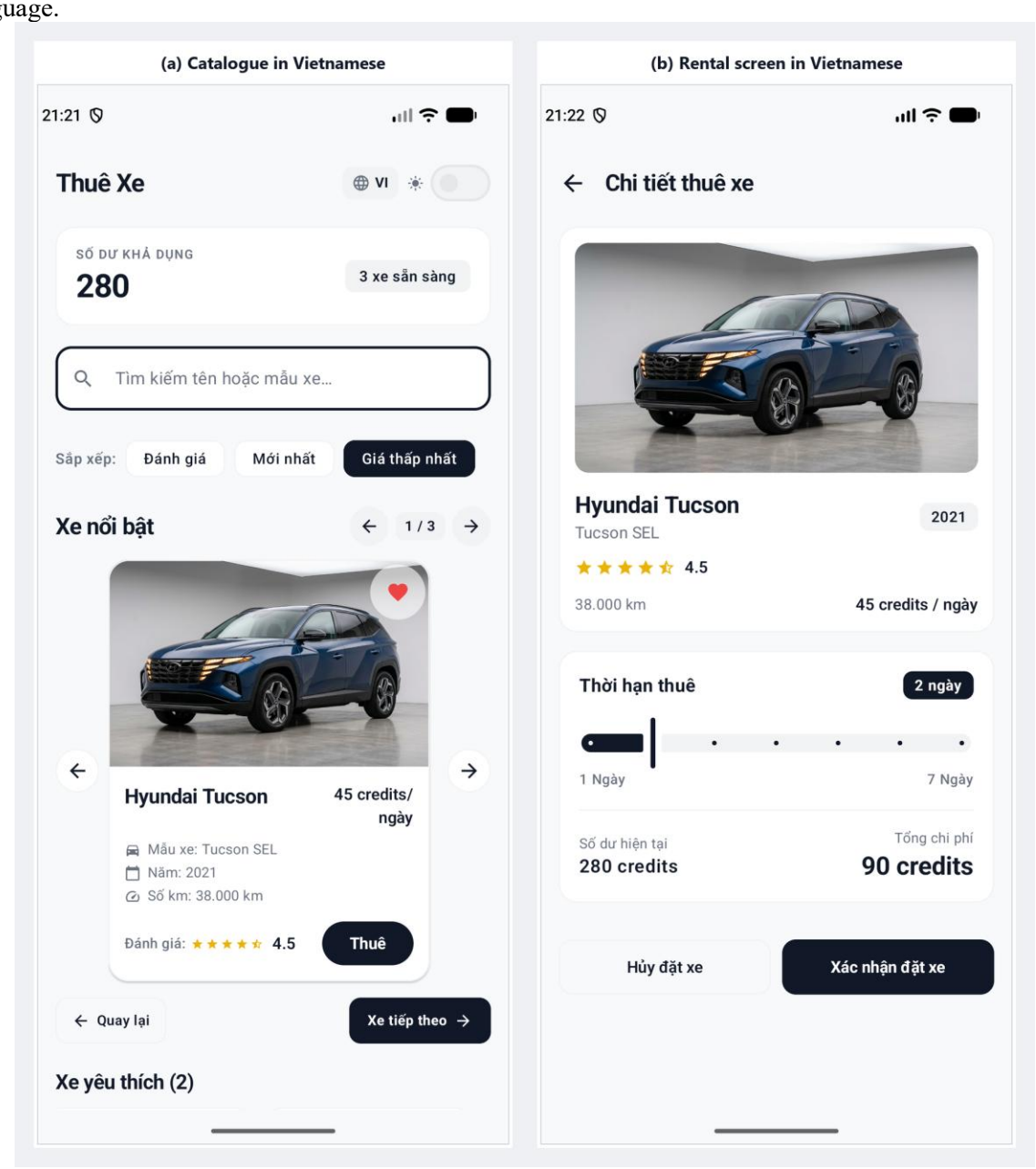


Figure 8. Vietnamese localisation on both screens.

4.8 Adaptive portrait and landscape layout

DriveEase does not lock the orientation. Each screen reads the current Configuration through LocalConfiguration and chooses a layout when it composes: portrait keeps the single-column flow described above, while landscape rearranges the same composables into columns. Both layouts share one

state and one set of widgets, so there is no second screen to maintain.
On the catalogue screen the credits card and the search and sort controls share the top row, the Popular Cars row widens each card to 300dp, and Favourites and Active Rentals form a side-by-side dashboard (Figure 9). On the rental screen the vehicle summary fills the left column while the duration slider, cost breakdown and action buttons fill the right column, so the total cost stays visible while the slider moves (Figure 10).
Rotation recreates the Activity. The slider position, validation message and open language menu are kept with rememberSaveable, and the selected car index, credits, favourites and bookings live in the ViewModel and repository, which outlive the recreated Activity. The screenshots in Figure 9 were taken after a booking and a rotation, with the 3-day rental and the favourite still present.

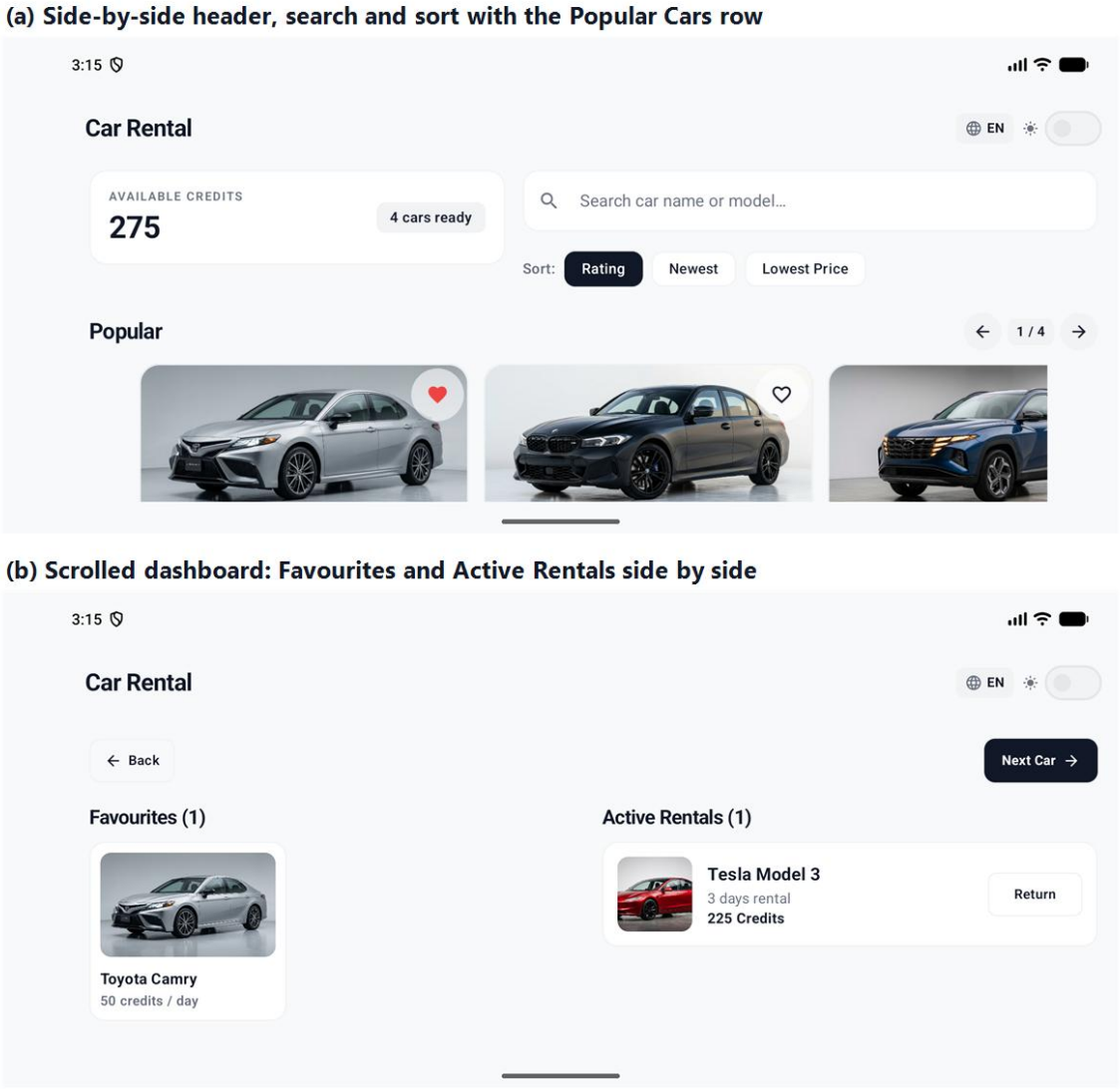


Figure 9. CatalogueScreen in landscape: (a) adaptive header and Popular Cars, (b) Favourites and Active Rentals side by side.

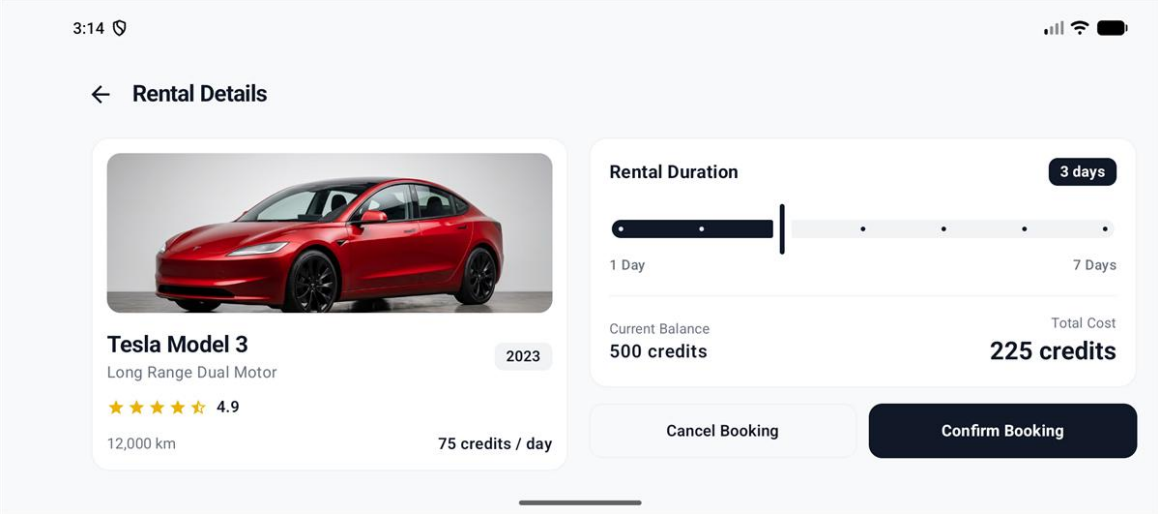


Figure 10. RentalScreen in landscape with the vehicle summary and booking controls in two columns.

5 System Architecture and Implementation

5.1 Architecture

The app uses five layers, each with one job. Dependencies point downwards, and only the UI knows about Compose.

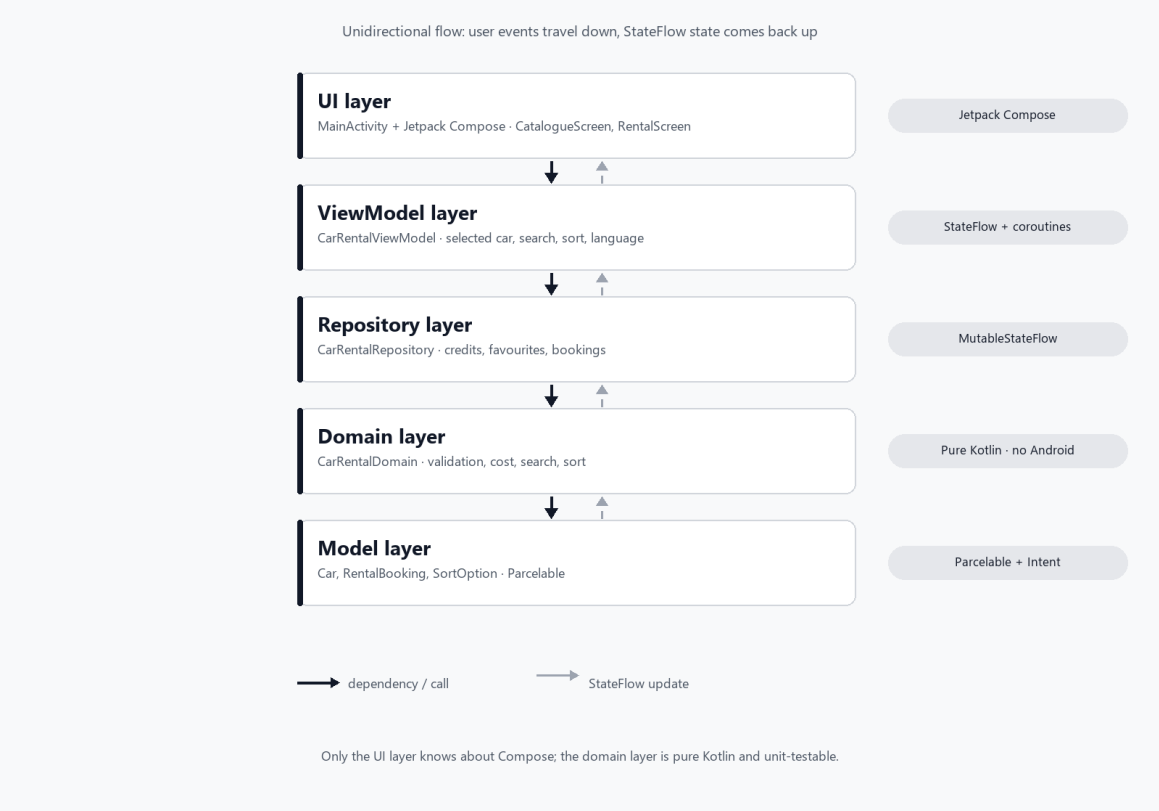


Figure 11. Layered architecture of DriveEase.

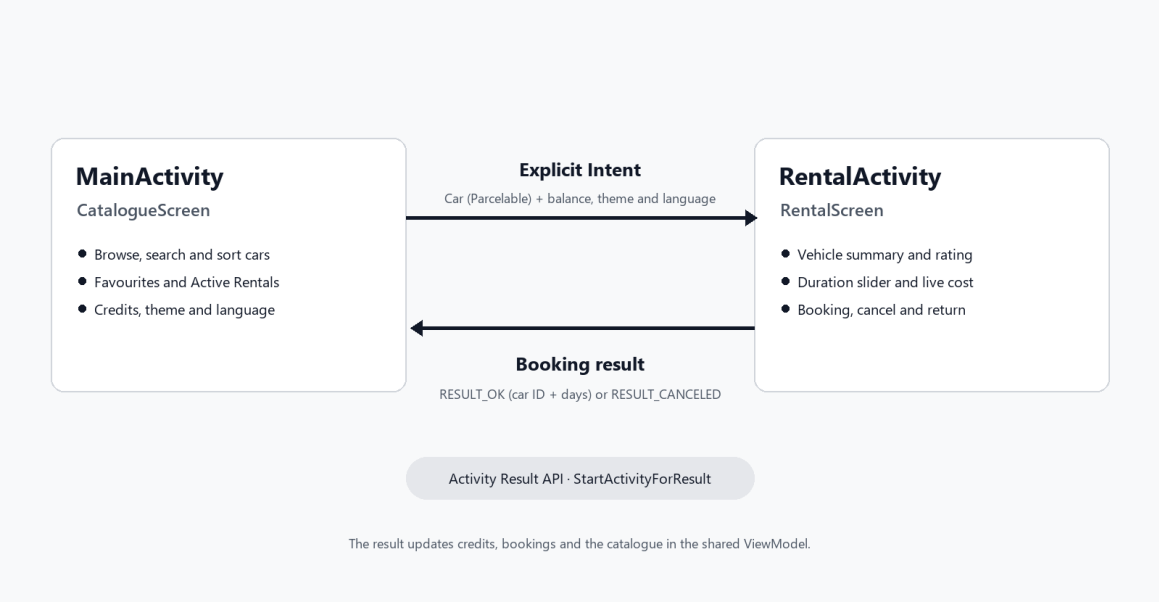


Figure 12. Navigation flow between the two activities.
MainActivity owns the ViewModel, launches RentalActivity with ActivityResultContracts.StartActivityForResult() (Android Developers, n.d.-a) and processes the returned result. RentalActivity receives the Car, validates the booking again with the same domain function, and returns the result.

5.2 Vehicle data model

Car is a Kotlin data class with 15 properties. Seven of them (engine, seats, colour, mileage, transmission, fuel type, description) have default values, so a car can be created with only the first eight. It implements Parcelable by hand (Android Developers, n.d.-f).

Property	Type	Meaning
id	String	Unique ID such as "car_001"
name, model	String	Brand name and model variant
year	Int	Manufacturing year
rating	Float	1.0 to 5.0
kmsDriven	Int	Kilometres driven
dailyCost	Int	Credits per day
imageResId	Int	Drawable resource ID
engine, color, mileage, transmission, fuelType, description	String	Specification text
seats	Int	Number of seats

The five vehicles used in the app:

Vehicle	Model	Year	Rating	Daily cost	Fuel
Toyota Camry	Camry SE	2022	4.8	50	Gasoline
Tesla Model 3	Long Range Dual Motor	2023	4.9	75	Electric
Hyundai Tucson	Tucson SEL	2021	4.5	45	Gasoline
BMW 3 Series	330i xDrive	2024	4.7	90	Premium gasoline
Honda Civic	Civic EX	2020	4.3	35	Gasoline

5.3 State management

CarRentalRepository keeps one MutableStateFlow per piece of state. The original vehicle list is never changed; the list shown on screen is rebuilt from it whenever search, sort or bookings change.

State	Type	Description
_creditBalance	MutableStateFlow<Int>	Current credits, starting at 500
_favourites	MutableStateFlow<Set<String>>	IDs of favourite cars
_rentedBookings	MutableStateFlow<List<RentalBooking>>	Active rentals
_searchQuery, _sortOption	MutableStateFlow	Current search text and sort choice
_availableCars	MutableStateFlow<List<Car>>	Derived: filtered and sorted cars still available
_favouriteCars	MutableStateFlow<List<Car>>	Derived: cars whose IDs are favourites

5.4 Activity communication with Intents

When the user taps "Rent", MainActivity builds an explicit Intent for RentalActivity and starts it through a launcher registered with rememberLauncherForActivityResult (Android Developers, n.d.-b; n.d.-c). I used the Activity Result API instead of the older startActivityForResult() because that method is deprecated and does not fit Compose.

Sending (MainActivity)

```
val intent = Intent(this, RentalActivity::class.java).apply {
    putExtra(RentalActivity.EXTRA_CAR, car)                // Parcelable
    putExtra(RentalActivity.EXTRA_CREDIT_BALANCE, viewModel.creditBalance.value)
    putExtra(RentalActivity.EXTRA_IS_DARK_MODE, isDarkMode)
    putExtra(RentalActivity.EXTRA_LANGUAGE, appLanguage)
}
rentalLauncher.launch(intent)
```

Receiving (RentalActivity)

```
val car = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    intent.getParcelableExtra(EXTRA_CAR, Car::class.java)
```

```
} else {
   @Suppress("DEPRECATION") intent.getParcelableExtra(EXTRA_CAR)
}
val creditBalance = intent.getIntExtra(EXTRA_CREDIT_BALANCE, 500)
```

Returning the result (RentalActivity) and handling it (MainActivity)

```
// RentalActivity
setResult(RESULT_OK, Intent().apply {
   putExtra(EXTRA_BOOKED_CAR_ID, car.id)
   putExtra(EXTRA_RENTAL_DAYS, days)
   putExtra(EXTRA_TOTAL_COST, totalCost)
})
finish()

// MainActivity
if (result.resultCode == RESULT_OK) viewModel.processBookingResult(bookedCarId, rentalDays)
else viewModel.notifyBookingCancelled()
```

Intent extra	Content
EXTRA_CAR	The full Car object (Parcelable)
EXTRA_CREDIT_BALANCE	Balance used for validation in RentalActivity
EXTRA_IS_DARK_MODE	Theme continuity
EXTRA_LANGUAGE	"en" or "vi"
EXTRA_BOOKED_CAR_ID, EXTRA_RENTAL_DAYS, EXTRA_TOTAL_COST	Result returned to MainActivity

Why Parcelable. Parcelable is Android's own mechanism for passing objects between components. It is faster than Java Serializable because it does not rely on reflection, and every field is written and read explicitly, so the format is visible in the code. The whole 15-property Car travels as one extra instead of 15 separate putExtra calls. On Android 13 and later the typed getParcelableExtra(key, Class) overload is used, with the older call as a fallback.

5.5 Rental rules and credit management

Each user starts with 500 credits. The cost of a rental is the daily cost multiplied by the number of days. validateRental() applies four rules in a fixed order and stops at the first failure.

```
fun validateRental(car: Car, rentalDays: Int, currentBalance: Int,
                   isAlreadyRented: Boolean = false): ValidationResult {
   if (isAlreadyRented) return ValidationResult.Error("...", R.string.error_already_rented)
   if (rentalDays !in MIN_RENTAL_DAYS..MAX_RENTAL_DAYS) return ValidationResult.Error("...",
R.string.error_duration)
   val totalCost = calculateTotalCost(car.dailyCost, rentalDays)
   if (totalCost > MAX_RENTAL_COST) return ValidationResult.Error("...", ...)   // 400
   if (totalCost > currentBalance) return ValidationResult.Error("...", ...)
   return ValidationResult.Success
}
```

1. The car is not already rented.
2. The duration is between 1 and 7 days.
3. The total cost is at most 400 credits.
4. The total cost is at most the available balance.

The 400 credit cap means the longest allowed rental depends on the car. It is the cap, not the 7 day limit, that restricts the two most expensive cars:

Vehicle	Daily cost	Cost for 7 days	Longest valid rental
Honda Civic	35	245	7 days
Hyundai Tucson	45	315	7 days
Toyota Camry	50	350	7 days
Tesla Model 3	75	525	5 days (375 credits)
BMW 3 Series	90	630	4 days (360 credits)

Cancellation. If the user presses Back or "Cancel Booking", RentalActivity calls setResult(RESULT_CANCELED) and finishes. MainActivity shows "Booking cancelled. No credits deducted." and nothing else changes.
Return. repository.returnRentedCar(carId) removes the booking from the rented list, puts the car back into the available list and refunds the full total cost.
Atomic update. processBooking() validates first and changes state only on success. A booking therefore deducts credits, records the booking and removes the car from the catalogue together, or does none of these.

5.6 Search and sorting

searchCars() checks, case-insensitively, whether the query is contained in the car name or model; an empty query returns every car. sortCars() returns a new list and never sorts the original in place. Each change to the search text or sort option resets the featured card to the first car, which fixed a bug described in Section 7.5.

Option	Kotlin operation	Order
RATING_DESC	sortedByDescending { it.rating }	Highest to lowest
YEAR_DESC	sortedByDescending { it.year }	Newest to oldest
COST_ASC	sortedBy { it.dailyCost }	Lowest to highest

The full pipeline is: canonical list, then remove rented cars, then search, then sort, then publish as availableCars.

5.7 Favourites

Favourites are stored as a set of car IDs. They are independent of availability: getFavouriteCars() looks cars up in the canonical list, not in the available list, so a rented car stays in the Favourites row.

5.8 Theme and language across activities

RentalActivity is a separate Activity with its own lifecycle, so it cannot observe MainActivity's ViewModel. The dark mode flag and language code are therefore passed in the Intent and applied once when it starts. MyApplicationTheme(darkTheme) picks between a light and a dark Material 3 colour scheme. A custom AppLocaleProvider creates a Context for the chosen language and supplies it through LocalContext, which is how the Vietnamese strings in res/values-vi/strings.xml are used without recreating the Activity.

5.9 Code structure and Kotlin features

The project follows the standard Android layout and separates packages by responsibility (the full tree is in Appendix A): model, domain, data, ui, ui/components and ui/theme. Kotlin features used to keep the code short and safe:

- Data classes with default and named arguments for Car and RentalBooking.
- An object (CarRentalDomain) with named constants for the limits (MIN_RENTAL_DAYS, MAX_RENTAL_DAYS, MAX_RENTAL_COST, INITIAL_CREDIT_BALANCE) instead of magic numbers.
- A result type with Success and Error cases, so callers must handle both.
- An enum for SortOption and immutable collection operations (filter, sortedBy) for derived lists.
- Null safety and the Elvis operator when reading Intent extras and Parcel values.
- StateFlow with collectAsState() so the UI recomposes automatically (Android Developers, n.d.-h).

6 Functionality and Technical Implementation

6.1 Requirements checklist

Requirement	Implementation	Test
Browse five vehicles	Popular Cars LazyRow with arrows and Back/Next buttons	TC01 to TC03
Open second Activity with an explicit Intent	"Rent" button starts RentalActivity	TC04
Transfer vehicle data with Parcelable	Car (15 properties) as EXTRA_CAR	TC05
Rental duration with a non-text widget	Material 3 Slider, 1 to 7 days, live cost	TC06
Rating with a non-text widget	Custom StarRatingBar (full, half, empty stars)	TC01
Credit rules (500 start, 400 cap, balance check)	CarRentalDomain.validateRental ()	TC06, TC09, TC10, UT-01 to UT-08
Confirm or cancel a booking	RESULT_OK or RESULT_CANCELED to MainActivity	TC06 to TC08
Favourites	Heart toggle and Favourites row	TC12, TC13, TC23, UT-18
Search and sort	searchCars() and sortCars()	TC14 to TC19, UT-09 to UT-13
Additional: dark mode	Switch and Material 3 colour schemes	TC20
Additional: English and	AppLocaleProvider and string	TC21

Requirement	Implementation	Test
Vietnamese	resources	
Additional: return a rented car	"Return" button and refund	TC22, UT-19

6.2 Browsing

The Popular Cars row gives the key detail of one car at a time and a swipeable overview of the rest. Moving with the arrows, swiping the row or tapping a card all update the same selected index, so the two views never disagree. The Popular Cars row uses rememberLazyListState so the row scrolls smoothly to the selected card.

6.3 Rental validation and feedback

Validation happens in RentalActivity before anything is returned. A failure becomes an inline banner rather than a toast, because a toast disappears after a few seconds while the user still needs the message in front of them as they move the slider. Toasts are used for events that are finished: booking confirmed, cancelled or returned.

Event	Feedback (English)
Booking successful	"Booking successful! {cost} credits deducted for {days} day(s)."
Booking cancelled	"Booking cancelled. No credits deducted."
Car returned	"{Car name} returned to catalogue. Credits refunded!"
Over 400 credits or insufficient balance	Inline error banner on the rental screen

Toast text is built with a LocalizedToastMessage (a string resource ID plus format arguments) and resolved in the Activity, so it follows the selected language even when the message starts in the ViewModel.

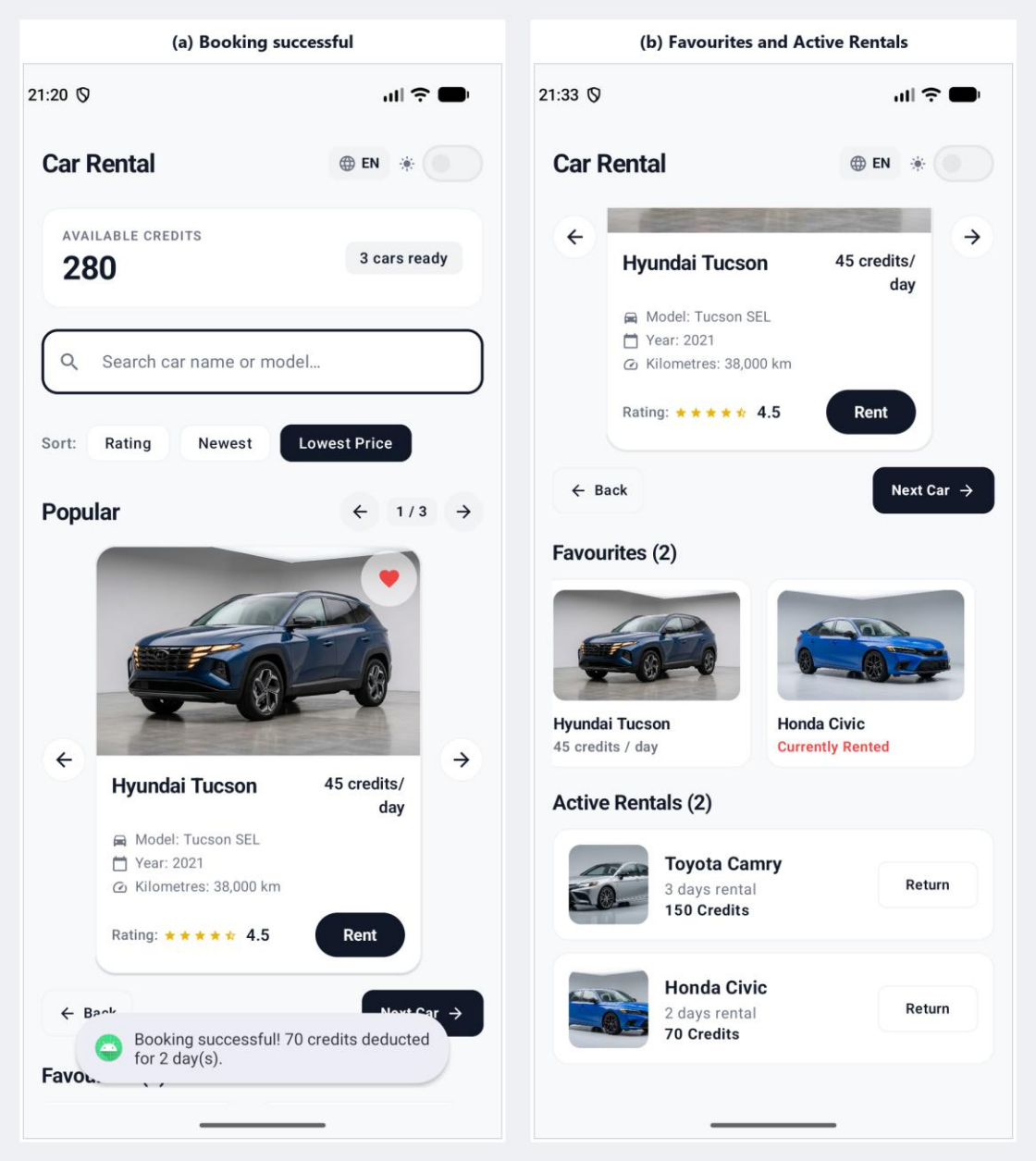


Figure 13. Booking successful toast and "Active Rentals" section.

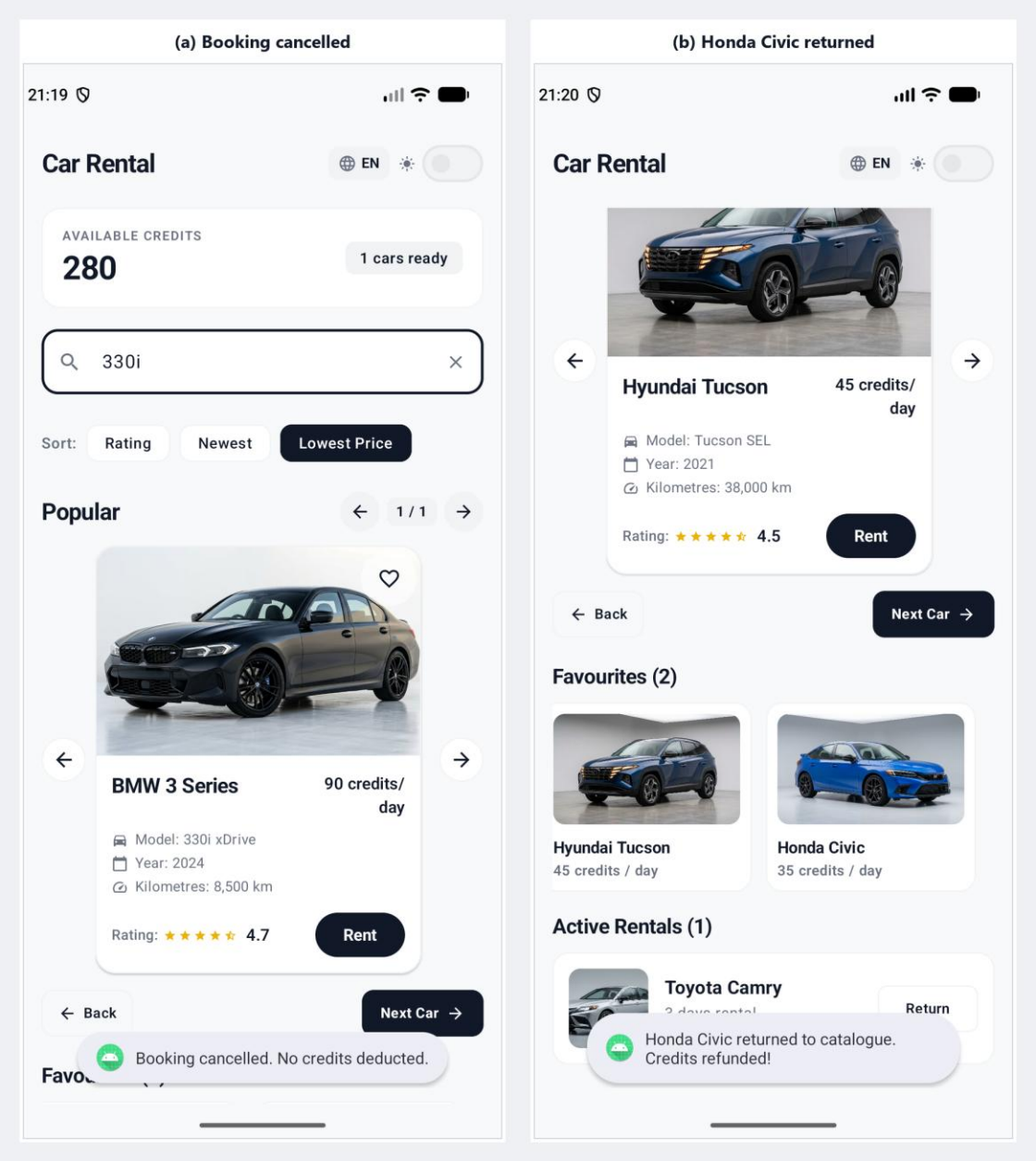


Figure 14. Booking cancelled toast and car returned toast.

6.4 Non-text widgets

Widget	Used for	Reason
StarRatingBar (custom, Material icons Star, StarHalf, StarOutline)	Showing the rating on cards and the rental screen	Shows 1 to 5 stars with half-star precision; replaces the XML RatingBar, which is not available in Compose
Slider	Choosing 1 to 7 days	Touch-friendly, cannot give an invalid value, gives live cost

Widget	Used for	Reason
		feedback
Switch	Light and dark mode	Clear two-state control
FilterChip	One-tap sort selection	Faster than opening a menu for each sort choice

7 Testing and Debugging

7.1 Strategy

Testing was done at three levels. Pure unit tests check the rules and state transitions on the JVM, with no emulator. Compose UI tests (3 cases in CarRentalUITest) check that the app launches, filters the list and toggles favourites on a device. Manual functional testing on the emulator then walks through each user story, including both languages and both themes. For the unit tests I used boundary value analysis: for each limit, one case on the boundary and one just outside it (for example 400 credits passes and 450 fails; 7 days passes and 8 fails).

7.2 Functional test cases (manual)

ID	Test	Action	Expected result	Actual result	Status
TC01	Launch	Open the app	Five vehicles; balance 500	Five vehicles; badge shows 500	Pass
TC02	Next vehicle	Tap right arrow	Next vehicle shown	Card and counter update	Pass
TC03	Previous vehicle	Tap left arrow	Previous vehicle shown	Card and counter update	Pass
TC04	Open rental screen	Tap "Rent"	RentalActivity opens for that car	Correct car displayed	Pass
TC05	Parcelable transfer	Select BMW 3 Series, tap Rent	BMW 3 Series opens	Name, model, year and cost correct	Pass
TC06	Valid booking	Camry, 3 days, Confirm	150 credits deducted; car removed	Balance 350; Camry under Active Rentals	Pass
TC07	Cancel button	Tap "Cancel Booking"	No change	Balance unchanged; car available	Pass
TC08	Cancel with Back	Press system Back	No change	Balance unchanged; car	Pass

ID	Test	Action	Expected result	Actual result	Status
				available	
TC09	400 credit cap	BMW, 5 days (450)	Booking blocked	Banner: exceeds 400 credit limit	Pass
TC10	Insufficient balance	Reduce balance with rentals, rent again	Booking blocked	Banner: insufficient credit balance	Pass
TC11	Duplicate booking	Look for a rented car in the catalogue	Not rentable again	Rented car not in available list	Pass
TC12	Add favourite	Tap heart	Appears in Favourites row	Heart filled; row appears	Pass
TC13	Remove favourite	Tap heart again	Removed from row	Heart outline; row hides if empty	Pass
TC14	Search by name	Type "Tesla"	Only Tesla Model 3	Only Tesla shown	Pass
TC15	Search by model	Type "330i"	Only BMW 3 Series	Only BMW shown	Pass
TC16	Clear search	Delete the text	All vehicles	Full catalogue back	Pass
TC17	Sort by rating	Rating, highest first	Tesla (4.9) first	Ordered by rating	Pass
TC18	Sort by year	Year, newest first	BMW (2024) first	Ordered by year	Pass
TC19	Sort by cost	Cost, lowest first	Honda (35) first	Ordered by cost	Pass
TC20	Dark mode	Turn on the Switch, open Rent	Both screens dark	Black background on both	Pass
TC21	Language	Tap globe icon	Vietnamese UI	Labels, buttons and toasts in Vietnamese	Pass
TC22	Return car	Tap "Return"	Car back; credits refunded	Car in catalogue; balance restored	Pass
TC23	Favourite then rent	Favourite a car, rent it	Stays in Favourites row	Visible in row and in Active Rentals	Pass

7.3 Automated unit tests

There are 27 JUnit tests: 13 in CarRentalDomainTest, 8 in CarRentalRepositoryTest and 6 in CarRentalViewModelTest. The two newest ViewModel tests check that the selected car index is stored in the ViewModel and clamped to the available range, which is what keeps the current card stable across a rotation. They run with ./gradlew app:testDebugUnitTest.

ID	Area	What is checked	Result
UT-01	Cost	1-day and 7-day cost at 50 credits per day	Pass
UT-02	Duration	1 and 7 days are accepted	Pass
UT-03	Duration	0 and 8 days are rejected	Pass
UT-04	400 cap	Exactly 400 (80 x 5) is accepted	Pass
UT-05	400 cap	450 (90 x 5) is rejected	Pass
UT-06	Balance	200 credit rental with balance 250 is accepted	Pass
UT-07	Balance	200 credit rental with balance 150 is rejected	Pass
UT-08	Duplicate	Already rented car returns an error	Pass
UT-09	Sort	RATING_DESC order: Tesla, Toyota, BMW, Hyundai, Honda	Pass
UT-10	Sort	YEAR_DESC order: BMW, Tesla, Toyota, Hyundai, Honda	Pass
UT-11	Sort	COST_ASC order: Honda, Hyundai, Toyota, Tesla, BMW	Pass
UT-12	Immutability	Sorting does not change the original list	Pass
UT-13	Search	Case-insensitive name and model filter; empty query returns all	Pass
UT-14	Repository	Initial state: balance 500, 5 available, 0 rented	Pass
UT-15	Repository	Camry for 3 days: balance 350, 4 available	Pass
UT-16	Repository	Second booking of the same car fails; balance	Pass

ID	Area	What is checked	Result
		unchanged	
UT-17	Repository	Cancelled booking leaves credits unchanged	Pass
UT-18	Repository	Rented car stays in favourites	Pass
UT-19	Repository	Returned car restores credits and availability	Pass
UT-20	Repository	nextCar() moves index from 0 to 1	Pass
UT-21	Repository	resetState() restores 500 credits, 5 available, 0 rented	Pass
UT-22	ViewModel	Default language is English	Pass
UT-23	ViewModel	Switching to Vietnamese updates the language	Pass
UT-24	ViewModel	Toggling the language alternates between English and Vietnamese	Pass
UT-25	ViewModel	Invalid language code is ignored	Pass
UT-26	ViewModel	Setting the current car index updates the selected car	Pass
UT-27	ViewModel	Current car index is clamped to the available range	Pass

Result: 27 of 27 tests passed, 0 failures.

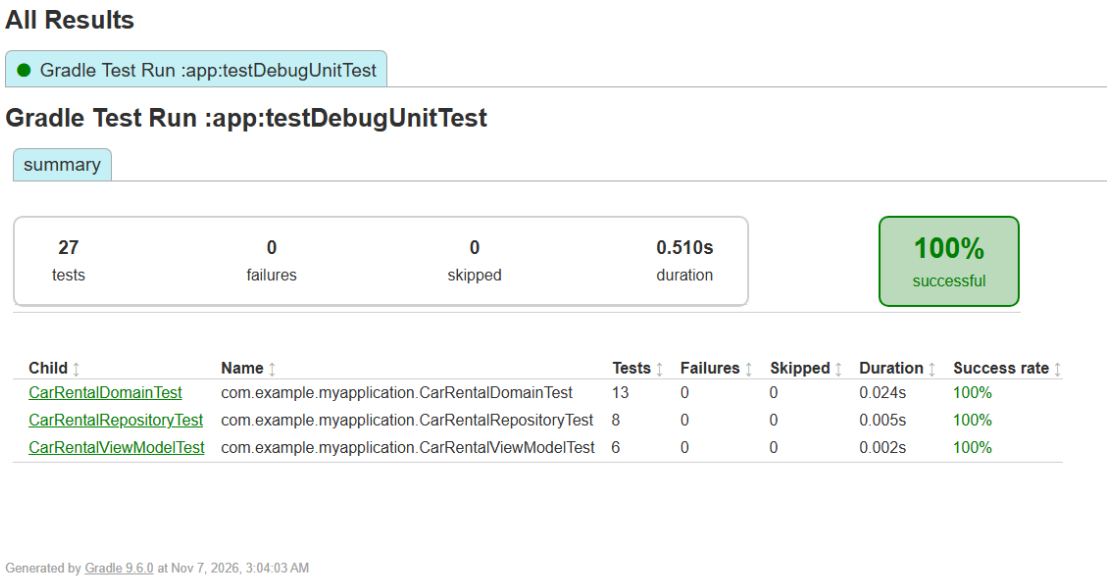


Figure 15. Unit test results in the Gradle test report.

7.4 Debugging

Five defects were found while testing, each traced to a root cause and re-tested after the fix.

Symptom	Root cause	Fix	Re-test
Catalogue did not refresh after a booking or a return	availableCars was not re-emitted after _rentedBookings changed	Call updateComputedCars() at the end of processBooking() and returnRentedCar()	TC06, TC22
Next button jumped to a wrong card after a search	The selected index was not reset when the filtered list became shorter	Reset the index to 0 in setSearchQuery() and setSortOption()	Searched "Tesla" (1 result), pressed Next: stayed at 0
Favourites row did not update when a heart was tapped	_favouriteCars was not recomputed after _favourites changed	Call updateFavouriteCars() inside toggleFavourite()	TC12, TC13
RentalActivity crashed on API below 33	getParcelableExtra(key, Class) exists only from API 33	Version check with the deprecated overload as fallback	Ran on API 24 emulator: no crash
Toasts stayed in English in Vietnamese mode	The ViewModel built the message from hard-coded English text	LocalizedToastMessage(resId, formatArgs), resolved in the Activity	TC21

8 Challenges, Explorations and Takeaways

8.1 Passing a 15-property object between activities

Challenge. The Car model has 15 properties. Sending each as a separate extra would make the Intent long and easy to break, because the sender and receiver must agree on every key and type.
Exploration. I compared Serializable and Parcelable. Serializable needs no extra code but uses reflection, which is slower on Android. I also considered the @Parcelize annotation, but the kotlin-parcelize plugin was not set up in the Gradle version catalog, and I preferred not to change the build configuration for one class. I wrote Parcelable by hand.
Outcome and takeaway. The first working version passed every field correctly. The part that needed care was the order: createFromParcel() must read fields in exactly the order writeToParcel() wrote them, because a mismatch gives wrong values with no error. Later I also found the API 33 crash (Section 7.4), which taught me that code that works on my emulator may fail on older Android versions.

8.2 Keeping credits, bookings and the catalogue consistent

Challenge. One booking changes four things at once: the balance, the bookings list, the available cars and the selected index. If any one is skipped, the app contradicts itself, for example credits taken while the car is still shown as available.
Exploration. Keeping variables inside the Activity would lose them on rotation and could not be shared. A companion object would work but is fragile and hard to test. A ViewModel with StateFlow survives configuration changes, is reactive and can be tested without a device.
Outcome and takeaway. I made processBooking() the only place that changes booking state. It validates through CarRentalDomain first and changes state only on success. The two bugs where the catalogue and the favourites row did not refresh (Section 7.4) had the same cause: derived state was not recomputed after a source changed. Separating validation from state change made both parts easy to test, and it is the main design lesson of this project.

8.3 Search and sort without losing the original list

Challenge. Search and sort must change what is shown, but clearing the search has to bring back the full list in the original order.
Exploration. Sorting the displayed list directly is simple, but it loses the original. I kept one canonical list and derived the displayed list from it each time using Kotlin's non-mutating functions.
Outcome and takeaway. A test (UT-12) confirms that sorting leaves the original unchanged. Using sortedBy and filter instead of in-place sorting removed a whole class of bugs. The search bug in Section 7.4 came from a different cause: the selected index was not reset when the list got shorter, so the derived list was correct but the index pointed past its end.

8.4 Theme consistency across two activities

Challenge. Both activities must show the same theme, but RentalActivity cannot observe MainActivity's ViewModel.
Exploration. Following the system theme would remove the user's control. An application-scoped ViewModel would work but ties the two activities together more than necessary. Passing a Boolean in the Intent is simple and fits how Android activities already communicate.
Outcome and takeaway. RentalActivity is short-lived, so reading the theme once at startup is enough. I learned that not every piece of state needs to be reactive; the right scope for state depends on how long its consumer lives.

8.5 Changing language at runtime

Challenge. Android normally picks the language at launch from device settings. I wanted a button that changes language immediately and also translates toasts.
Exploration. AppCompatDelegate.setApplicationLocales() needs API 33 or the AppCompat library, and this project is a pure Compose project with minimum SDK 24. Instead I built AppLocaleProvider, which wraps the content in a Context created with createConfigurationContext() and exposes it through LocalContext. The first version still showed English toasts, because the ViewModel created the text itself. Moving to LocalizedToastMessage fixed it.
Outcome and takeaway. This showed me how CompositionLocal passes values through a UI tree without adding a parameter to every function, and that user-visible text should be resolved as late as possible, close to the UI, not in business logic.

8.6 Redesign after using the first version

The first catalogue (20 October) showed one car at a time with Next and Back buttons. After using it, I found that comparing cars meant tapping repeatedly and that it felt dated next to the premium look I wanted. On 2 November I redesigned it: monochrome theme, pure black dark mode, larger photo-led cards and a swipeable Popular Cars row, with arrows, a counter and explicit Back/Next Car buttons for people who prefer tapping. The commit history shows this change (79436b7, a7451fc, e179a8e), and the fix 86ef3fd followed straight away when the new row exposed the index bug. The lesson was that a design should be judged by using it, not by whether the code works.

8.7 Overall reflection and future work

This assignment taught me to think about how features affect each other, not only how each screen works. Moving from XML layouts to Compose was hard at first, particularly state hoisting and when recomposition happens, but the result is shorter and reacts to state automatically.
Three improvements would come next. First, persistence: bookings, balance, favourites and language are lost when the app closes, so Room and DataStore would be the production choice. Second, more UI tests: the three Compose tests only cover launch, search and favourites, and a complete rental flow and theme switch should be tested too. Third, a drawn icon set and tablet layout, since the layout was only tested on phone-size screens.

8.8 Use of generative AI

Generative AI was used as a development and learning aid, not as a replacement for my own work. I used Google Antigravity with Gemini 3.8 Flash during October and November 2026 to explain Android and Compose concepts, check my understanding of the assignment logic, and draft parts of the Kotlin code and test structure. I also used an AI image-generation tool to create the five vehicle images shown in the catalogue and on the rental screen. Every output was reviewed, edited, built and tested before it was kept.
The logic came first. When I was unsure about a mechanism, I asked for an explanation before asking for code: why the selected car index must live in the ViewModel so it survives a rotation, how a 15-property Car travels through an Intent as a Parcelable, and why filtering and sorting should return a new list instead of changing the repository list. Code suggestions were treated as drafts; I read each change, ran ./gradlew app:testDebugUnitTest and the app on the emulator, fixed the defects listed in Section 7.4 and re-tested the affected flows.

- **Learning and planning:** explanations of the Activity lifecycle, Intents, StateFlow and configuration changes, which I re-explained in my own words and then applied in the app.
- **Code and debugging:** draft Kotlin and Compose functions, Gradle fixes and JUnit test structure, checked by building the project, running the 27 unit tests and repeating the manual test cases.
- **Vehicle images:** five AI-generated car images used on the cards and rental screen; I checked the model, body style and colour, cropped each image to the card ratio and stored it as a drawable resource.
- **Report support:** outline, test plan and wording suggestions, rewritten in my own words and checked against the app and the cited sources.

The vehicle images are illustrative assets, not manufacturer press photographs, and the specification data in Car.kt is written by hand and does not depend on them. All AI-assisted code, text and images were reviewed and validated by me. I understand and can explain the submitted code, can reproduce the test results, and accept responsibility for the final report. A longer record of the prompts and outputs is kept in the project documentation.

9 Conclusion

DriveEase meets the assignment brief with two activities that exchange data through explicit Intents, a 15-property Parcelable Car, a Slider and a custom star rating as non-text widgets, and rental rules that enforce a 500 credit start, a 400 credit cap and a balance check. Search, sort, favourites, dark mode, bilingual support and returning a rented car are added on top, and each has test cases.
The layered structure (model, domain, repository, ViewModel, UI) kept the rules testable: 27 unit tests pass, and the manual test table covers every user-facing feature. The main limitation is that data is only in memory. The main lesson is that consistent state, and a clear owner for each piece of it, matters more than any single screen.

10 References

Android Developers. (n.d.-a). *Getting a result from an activity*.
https://developer.android.com/training/basics/intents/result
Android Developers. (n.d.-b). *Intents and intent filters*.
https://developer.android.com/guide/components/intents-filters
Android Developers. (n.d.-c). *Jetpack Compose overview*. https://developer.android.com/compose
Android Developers. (n.d.-d). *Kotlin coroutines on Android*.
https://developer.android.com/kotlin/coroutines
Android Developers. (n.d.-e). *Material Design 3 for Compose*.

https://m3.material.io/develop/android/jetpack-compose
Android Developers. (n.d.-f). *Parcelable interface*.
https://developer.android.com/reference/android/os/Parcelable
Android Developers. (n.d.-g). *Slider (Material 3)*.
https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#Slider
Android Developers. (n.d.-h). *StateFlow and SharedFlow*.
https://developer.android.com/kotlin/flow/stateflow-and-sharedflow
Android Developers. (n.d.-i). *ViewModel overview*.
https://developer.android.com/topic/libraries/architecture/viewmodel
JetBrains. (n.d.). *Collection ordering*. Kotlin Documentation. https://kotlinlang.org/docs/collection-ordering.html
Nguyen, H. M. (2026). *DriveEase: car rental app interface design* [Figma file].
https://www.figma.com/design/54FhaO2u7iPpxt5mYDEJw6/Untitled?node-id=1-2
Nielsen, J. (1994). *10 usability heuristics for user interface design*. Nielsen Norman Group.
https://www.nngroup.com/articles/ten-usability-heuristics/
World Wide Web Consortium. (2018). *Web content accessibility guidelines (WCAG) 2.1*.
https://www.w3.org/TR/WCAG21/

Appendices

Appendix A: Project structure

app/src/main/java/com/example/myapplication/
MainActivity.kt Catalogue host, launches rentals, handles result
RentalActivity.kt Rental host, validates, returns result
data/CarRentalRepository.kt StateFlow state, atomic bookings
domain/CarRentalDomain.kt Validation, cost, search, sort (pure Kotlin)
model/Car.kt Parcelable vehicle
model/RentalBooking.kt Parcelable booking
model/SortOption.kt Sort enum
ui/CarRentalViewModel.kt UI state and actions
ui/components/StarRatingBar.kt Custom rating widget
ui/components/AppLocaleProvider.kt Runtime locale override
ui/theme/Color.kt, Theme.kt, Type.kt
app/src/test/.../CarRentalDomainTest.kt 13 tests
app/src/test/.../CarRentalRepositoryTest.kt 8 tests
app/src/test/.../CarRentalViewModelTest.kt 6 tests
app/src/androidTest/.../CarRentalUITest.kt 3 Compose UI tests
app/src/main/res/values/strings.xml, values-vi/strings.xml

Appendix B: Running the tests

./gradlew app:testDebugUnitTest
Expected result: BUILD SUCCESSFUL, 27 tests completed, 0 failed. A screenshot of this run is Figure 15 in Section 7.3.

Appendix C: Colour palette

Mode	Background	Surface (cards)	Primary accent
Light	#F8F9FA off-white	#FFFFFF	#111827 deep charcoal

Mode	Background	Surface (cards)	Primary accent
Dark	#000000 true black	#121214	#FFFFFF

Appendix D: Key design decisions

- **Compose instead of XML:** collectAsState() recomposes automatically when a StateFlow changes, so no adapter refresh calls are needed.
- **Activity Result API instead of startActivityForResult():** the old method is deprecated and does not work with Compose.
- **Domain logic separate from the repository:** CarRentalDomain has no Android dependencies, so its 13 tests run on the JVM.
- **RentalBooking is Parcelable:** it is only used inside MainActivity now, but this allows passing bookings between screens later.
- **True black dark theme:** better battery life on OLED screens and stronger contrast with the cards.